D D M M Y Y Y Y

ASSINMENT-02

1] What is Normalization? Explain the 1NF, 2NF, 3NF & BNF with examples

Normalization is the process of decomposing unsatisfactory relation schemas by breaking up their attributes into smaller relation schemas that possess desirable properties

> The goal of Normalization is to create a set of relational tables that are free of redundant data and can be consistently and correctly modified

> All tables in a relational database should ideally be in the third normal form (3NF)

The most commonly used Normal Forms.

- First Normal form (1NF)
- Second Normal form (2NF)
- Third Normal form (3NF)
- Boyce-Codd Normal form (BCNF)

First Normal form (1NF)

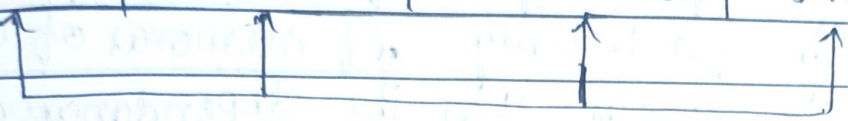
First Normal form is now considered to be part of the formal definition of relation historically, It was defined to disallow multivalued attribute composite attributes, and their combinations. It states that the domains of attributes must include only atomic (simple, indivisible) values and that the value of my attribute in a tuple must be a single value from the domain of that attribute.

D D M M Y Y Y Y

a)

DEPARTMENT

DName	Dnumber	Dmgr_SSN	Dlocations



(b)

DEPARTMENT

DName	DNumber	Dmgr_SSN	Dlocations
Research	5	333445555	{Bellaire, Sugarland, Houston}
Administration	4	987654321	{Stafford}
Headquarters	1	888665555	{Houston}

(c)

Department

DName	DNumber	Dmgr_SSN	Dlocation
Research	5	333445555	Bellaire
Research	5	333445555	Sugarland
Research	5	333445555	Houston
Administration	4	987654321	Stafford
Headquarters	1	888665555	Houston

a) A relation schema that is not in 1NF.

b) Sample state of relation DEPARTMENT

c) 1NF version of the same relation with redundancy.

Practice Rule: Eliminate Repeating Groups, i.e make a saparate table for each set of related attributes, and give each table a Primary redundancy.

D D M M Y Y Y Y

⇒ Second Normal form (2NF)

Second Normal form is based on the concept of fully functionally dependency. A functional dependency $X \rightarrow Y$ is fully functionally if removal of any attribute A from X means that the dependancy does not hold anymore.

A relation schema is in 2NF if every nonprime attribute in the relation is fully functionaly dependent on the Primary key of the relation.

Example:-

Original relation EP with key {Sid, course#}

Sid	course	HOURS	Proffessor	Position

3) Third Normal form (3NF)

Third normal form is based on the concept of transitive dependency. A functional dependency $X \rightarrow Y$ in a relation is a transitive dependency if there is a set of attributes Z that is not a subset of any key of the relation. (a) X is a super key of the relation.

example or (b) A is a prime attribute of the relation.

Ename	SSn	Bdate	Address	Dnumber	Dname	Dmgr-ssn

3NF Normalization

ED1

Ename	SSN	Bdate	Address	Dnumber

ED2

Dnumber	Dname	Dmgr-ssn

D D M M Y Y Y Y

Boyce-Codd Normal form (BCNF):

A relation schema R is in Boyce-Codd Normal form (BCNF) if whenver an FD $x \rightarrow A$ holds in R, then x is a superkey of R. Each normal form is strictly stronger than the previous one.

- Every 2NF relation is in 1NF
- Every 3NF relation is in 2NF
- Every BCNF relation is in 3NF
- There exist relations that are in 3NF but not in BCNF

2] what is functional Dependency? write algorithm to find minimal cover for set of functional Dependency.

A functional Dependency $x \rightarrow y$ states that for any two tuples in a relation, if they have the same value for attribute x, they must also have the same value for attribute y. it represents a constraint between attributes.

Properties:

- If $x \rightarrow y$ and $y \rightarrow z$, then $x \rightarrow z$ by transitivity.
- Used to identify keys and normalize relations

Algorithm for minimal cover:

Reduce the set of FDs to the smallest equivalent set.

Steps:

1. RHS Reduction: convert all FDs to have single attribute on RHS

Ex: $A \rightarrow BC$ becomes $A \rightarrow B$ & $A \rightarrow C$

D D M M Y Y Y Y

2. Remove extraneous LHS Attributes:
For each FD $X \to Y$ and each attribute A in X, check if $(F - \{X \to Y\}) \cup \{(X - A) \to Y\}$ logically implies $X \to Y$.
If yes, remove A from X.
Ex: If $AB \to C$ and $A \to C$ also holds, then B is extraneous.

3. Remove Redudant FDs:
For each FD $X \to Y$ in F, check if $F - \{X \to Y\}$ still implies $X - Y$.
If yes, delet $X \to Y$ from F

Example:-
Given $F = \{A \to BC, B \to C, A \to B, AB \to C\}$
Step 1: $F = \{A \to B, A \to C, B \to C, A \to B, AB \to C\}$
Step 2: $AB \to C$ has B extraneous because $A \to C$ exists → remove to $A \to C$
Step 3: $A \to B$ & $B \to C$ implies $A \to C$, so $A \to C$ is redundant → remove
minimal cover = $\{A \to B, B \to C\}$

3] Illustrate Insert, delete, Update, alter & drop commands in SQL.
DDL - Data Defination Language: Defines structure
DML - Data manipulation Language: modifies data

1] Insert - DML
Adds new rows into a table

Syntax: INSERT INTO table_name VALUES('val1', val2, ...);

Example: INSERT INTO student VALUES(101, 'ROOPA', 'CSE', 8.5);

Insert specific columns: INSERT INTO student (RollNo, Name) values(102, 'Ravi');

2. DELETE - DML

Removes existing rows based on condition.

Syntax: DELETE FROM table_name WHERE condition;

Example: DELETE FROM student WHERE RollNO = 101;

DELETE FROM student; → deletes all rows but keeps table structure

3. UPDATE - DML

modifies existing data

Syntax: UPDATE table_name SET col = val WHERE condition;

Example: UPDATE student SET marks = 9.0 WHERE RollNO = 102;

4. ALTER - DDL

modifies table structure without deleting data.

- Add column: ALTER TABLE student ADD Email VARCHAR(30);
- DROP column: ALTER TABLE student DROP column marks;
- Modify column: ALTER TABLE student MODIFY name VARCHA(50);
- Rename column: ALTER TABLE student

be inferred results in the complete set of all possible dependencies that can be inferred from F. In other words, given a set of FDs, all implied FDs can be derived using these three rules.

closure of a set of functional Dependencies: Given a set x of FDs in relation RR, the set of all FDs that are implied by x is called the closure of x, denoted as x.

Algorithms for Determining x:

x+ := x;
Repeat
oldx+ := x+;
for each FD Y→Z in F do
if Y ⊆ x+ then x+ := x+ ∪ Z;
until old x+ = x+;

Example: Given the set of functional dependencies:

A → BC
E → CF
B → E
CD → EF

Compute A, B+ of the set of attributes under this set of FDs.

Soln:

Step 1:

{A, B}+ := {A, B}.

Go round the inner loop 4 times, once for each of the given FDs.

D D M M Y Y Y Y

On the first Iteration, for $A \rightarrow BC$: $A \subseteq \{A, B\}$
$A, B+ := \{A, B, C\}$.
Step2:
On the second iteration, For $E \rightarrow CF$, $C \notin \{A, B, C\}$
Step3:
On the third iteration, for $B \rightarrow E$: $B \subseteq \{A, B, C\}$,
$A, B^+ := \{A, B, C, E\}$.
Step4:
On the fourth iteration, for $CD \rightarrow EF$, EF remain unchanged.
Go round the inner loop 4 times again.
On the first iteration, the result does not chang; on the second, It expands to $\{A, B, C, E, F\}$; on the thi and fourth, it does not change, Now go round the inner loop 4 times. the closure does not change, and so the whole process terminates, with
$AB+ = \{AB, C, E, F\}$.

Example:
Given
F = {SSN ENAME, PNUMBER → {PNAME, PLOCATION}, {SSN, PNUM
→ HOURS}
{SSN}+ = {SSN, ENAME}
{PNUMBER}+ = ?
{SSN, PNUMBER}+ = ?

5] Illustrate DDL alter command with an examples

ALTER is a DDL command used to modify the structure of an existing table without deleting data.

operations performed by ALTER:

1. ADD: Add new column.

Syntax: ALTER TABLE table_name ADD column_name datatype;

Example: ALTER TABLE Student ADD Email VARCHAR(30);

2. DROP: Remove an existing column.

Syntax: ALTER TABLE table_name DROP COLUMN column_name;

Example: ALTER TABLE Student DROP COLUMN Email;

This deletes the column & all data in it from the table.

3. MODIFY: change data type or size of a column

Syntax: ALTER TABLE table_name MODIFY COLUMN column_name new_datatype;

Example: ALTER TABLE Student MODIFY COLUMN Email VARCHAR(50);

This changes the size of Email from VARCHAR(30) to VARCHAR(50).

4. RENAME COLUMN: change the name of a column.

Syntax: ALTER TABLE table_name RENAME COLUMN old_name TO new_name;

Example: ALTER TABLE student RENAME TO
Student Info;

6. ADD CONSTRAINT: Add a constraint to a column
Syntax: ALTER TABLE table_name ADD CONSTRAINT
constraint_name &
constraint_type (column_name);
Example: ALTER TABLE Student ADD CONSTRAINT
PK_Student PRIMARY KEY (sid);

6] Explain the types of anomalies in SQL with an example
Anomalies are problems that occur in unnormalized tables due to redundancy
Insertion Anomaly: occurs when we cannot insert a fact because some part of the primary key is missing.
Example: In a table EMP_PROJ with key {EMPNO, ProjNo}, we cannot add a new project unless at least one employee is assigned to it.

Deletion Anomaly: occurs when deleting one fact unintentionally deletes another fact.
Example: In EMP_DEPT, if we delete the last employee of a department, we also lose information about the department itself.

Update Anomaly: occurs when the same data is repeated in multiple rows, so updating it requires changing all copies. If we miss one, the database becomes inconsistent. Example: changeng a Department name in EMP_DEPT requires updating all employees of that department

Explain Group By and HAVING Clause in SQL with Example.

GROUP BY clause groups rows that have the same values in the specified columns. It is used together with aggregate functions like COUNT, SUM, AVG, MAX, MIN to perform calculations on each group instead of on entire table.

HAVING clause is used to filter groups after aggregation. Unlike WHERE which filters rows before grouping, Having filters the result of GROUP BY

EXAMPLE:- To find the departments where the average marks of student is greater than 75, we group the STUDENT table by dept and apply AVG on marks, then use HAVING to keep only those groups where AVG(marks) > 75. WHERE would be used if we wanted to exclude students with marks below 40 before calculating the average

D D M M Y Y Y Y

8] Explain informal design guidelines for relational schema design.

⟹ Informal design guidelines are common-sense rules used by database designers to create good relational schemas that are easy to understand, maintain, and free from redundancy and anomalies.

The 4 main Guidelines:

1] clear Semantics of Attributes in a Relation.

> Each relation should represent one entity type or one relationship type.

Don't mix unrelated attributes in the same table.

Example: Don't put employee details and department details together in one big table. Keep them separate as EMP and DEPT. This makes meaning clear.

2] Reduce Redundancy and Avoid update Anamalies

Avoid repeating the same data in multiple rows. If data is repeated, updating it in one place but missing another causes inconsistency. Use functional dependencies to split tables and remove redundancy.

3] minimize NULL values in Tuples.

Design tables so that NULL values are rare. If many attributes are often NULL, it means those attributes don't apply to all tuples

split such tables so that each table stores only applicable attributes.

4] Avoid generating spurious Tuples on Natural join
Bad decomposition can cause fake/extra tuples when you join tables back.
Always use lossless join decomposition base on FDs.
This ensures that joining the decomposed tables gives back the original data without garbage tuples

9] Demonstrate Transaction states and Additional operations.
A transaction is an atomic unit of work that should either be completed in its entirety or not done at all.
For recovery purposes, the system needs to keep track of when each transaction starts, terminates, and commits, or aborts. therefore, the recovery manager of the DBMS needs to keep track of the following operations:
> BEGIN_TRANSACTION. This marks the beginning of transaction execution.
> READ or WRITE. these specify read or write operations on the database items that are executed as part of a transaction
> END_TRANSACTION: This specifies that READ and WRITE transaction operations have ended and marks the end of transaction execution

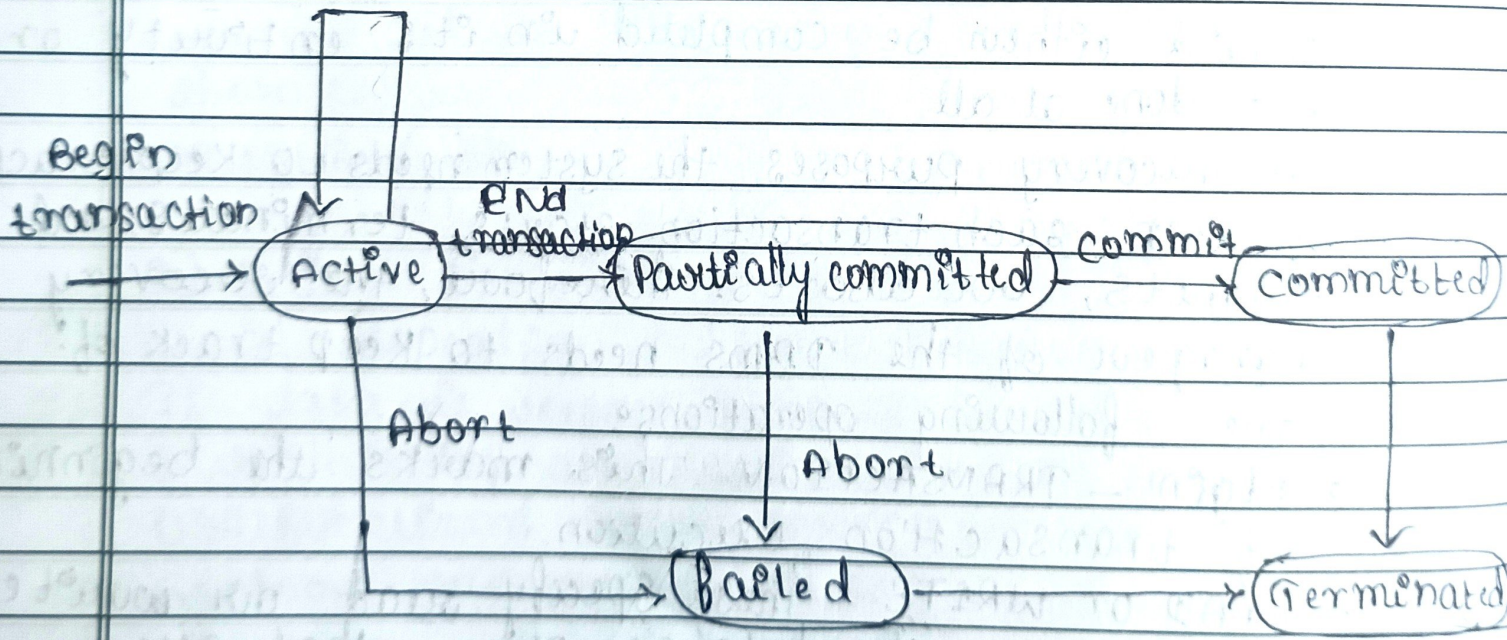
D D M M Y Y Y Y

However, as this point it may be necessary to check whether the changes introduced by the transaction can be permanently applied to the database (committed) or whether the transactions has to be aborted because it violates serializa_bility or for some other reason.

> COMMIT_TRANSACTION. This signals a successfull end of the transaction so that any changes (updates) executed by the transaction can be safely committed to the database and will not be undone.

> RollBack (or ABORT). This signals that the transa_ction has ended unsuccessfully, so that any changes or effects that the transaction may have applied to the database must be undone.

Read, write.



10) Demonstrate working of Assertion & Triggers in database? Explain with an example

In SQL, users can specify general constraints - those that do not fall into any of the categories - via declarative assertions, using the CREATE ASSERTION statement. Each assertion is given a constraint name and is specified via a conditions similar to the WHERE clause of an SQL query.

For Example:-

To specify the constraint name that and is specified via the salary of the manager of the department that the employee works for in SQL, we can write the following assertion

```
CREATE ASSERTION SALARY — CONSTRAINT.
CHECK (NO EXISTS (SELECT *
FROM EMPLOYEE E, EMPLOYEE M, DEPARTMENT D
WHERE E.Salary > M.salary AND E.Dno = D.Dnumber
  AND D.mgr_SSn = M.SSn));
```

⇒ Triggers in SQL

Another important statement is SQL is CREATE TRIGGER, In many cases it is convenient to Specify the type of action to be taken when Certain events occur and when certain conditions are satisfied.

For Example: it may be useful to specify a condition that, if violated, causes some user to be informed of the volation.

D D M M Y Y Y Y

RS: CREATE TRIGGER SALARY_VIOLATION
BEFORE INSERT OR UPDATE OF SALARY, SUPERVISOR
_SSN
ON EMPLOYEE FOR EACH ROW
WHEN (NEW.SALARY > (SELECT SALARY FROM EMPLOYEE
WHERE SSN = NEW.SUPERVISOR_SSN))
INFORM_SUPERVISOR (NEW.Supervisor_SSN, NEW.SSN);
A typical trigger which is regarded as an ECA (Event, condition, Action) rule has three components
1] The event: these are usually database update operations that are explicitly applied to the database.

2] the condition that determines whether the rule action should be executed

3] The action to be taken..

11] Explain the cursor & its properties in embedded SQL with an example.
A cursor is a database object that allows a host programming language like C or Java to process the result of a query row by row, instead of all at once.

Properties: A cursor can be Scrollable, allowing movement both forward and back-ward in the result set. It can be updatable, allowing update or delete of the current row. It can be read-only

preventing any modification.

operations: First we declare a cursor for a SELECT query. Then we open the cursor to execute the query. Next we fetch rows one by one into host variables. finally we close the cursor to release resource.

Example: In embedded SQL, we declare a cursor for selecting student names, open it, fetch each name in a loop, and close it when done. This allows row-by-row processing inside a program.

12] Demonstrate the system log in database instruction.

The system log is a sequential, append-only file kept on disk that records all transaction operations affecting database items, plus other transaction information needed for recovery from failures.

purpose:

To enable recovery from failures that effect transactions by keeping track of all changes to database items & transaction status.

1. Nature of the Log.

* The log is sequential, append-only file stored on disk.
* It is not affected by any type of failure

except disk or catastrophic failure

2. Log Buffer.
* main memory buffers called log buffer hold the last part of the log file.
* New log entires are first added to the log buffer in main memory

3. Types of Log Records.
In these entries, T refers to a unique transaction-id generated by the system for each transaction.

1. [start_transaction, T] - indicates that transaction T has started execution
2. [write_item, T, X, old_value, new_value] Indicates T has changed database item X From old_value to new_value. Used for UNDO/REDO during recovery.
3. [read_item, T, X] - Indicates T has read the value of data item X.
4. [commit, T] - Indicates T has completed successfully and its effect can be committed permanently to the database
5. [abort, T] - indicates transaction T has been aborted [commit] [T] [abort]

use in Recovery.
> For failed [aborted transactions: use old_value in write_item records to UNDO changes

13] Discuss ACID Properties of Database Transactions.

Transactions should possess several properties, often called the ACID properties; they should be enforced by the concurrency control and recovery methods of the DBMS. The following are the ACID properties.

→ Atomicity: A transaction is an atomic unit of Processing; it should be either be performed in its entirety or not performed at all.

→ consistency preservation: A transaction should be consistency preserving, meaning that if it is completely executed from beginning to end without interference from other transactions, it should take the database from the one consistent state to another

→ Isolation: A transaction should appear as though it is being executed in isolation from other transactions executing concurrently

→ Durability or permanency:-
The changes applied to the database by a committed transaction must persist in the database. These changes must not be lost because of any failure

14) Explain stored procedure language in SQL with an example.

A stored procedure is a precompiled collection of SQL statements stored in the database and executed by name. It can accept input parameters and perform complex logic.

Advantages: procedures are reusable, reduce network traffic because only the call is sent, improve security by following execution without direct table access, and execute faster due to precompilation.

Example: A procedure AddBonus can take employee ID and bonus amount as input, and update the salary of that employee. Users can call the procedure without knowing the internal UPDATE statement.
Stored procedures differ from functions because procedures can modify the database and need not return a value, while functions must return a value and are used in expressions.

15) Explain Two phase Locking protocol for concurrency control.*

A transaction is said to follow the two-phase locking protocol if all locking operations (read_lock, write_lock) precede the first unlock operation in the transaction

such a transaction can be decided into two phases. an expanding or growing (first) phase, during which new locks on items can be acquired but non can be released; and.
a shrinking (second) phase, during which existing locks can be released but no new locks can be acquired.
If lock conversion is allowed, then upgrading of locks (from read-locked to write-locked) must be done during the expanding phase, and downgrading of locks (from write-locked to read-locked) must be done in the shrinking phase

T_1	T_2
read_lock(Y);	read_lock(X);
read_item(Y);	read_item(X);
unlock(Y);	unlock(X);
write_lock(X);	write_lock(Y);
read_item(X);	read_item(Y);
X := X+Y;	Y := X+Y;
write_item(X);	write_item(Y);
unlock(X);	unlock(Y);

Initial values: X=20, Y=30
Result Serial schedule T_1
followed by ~~seau~~ T_2 : X=50, Y=80
Result of serial schedule T_2
followed by T_1 : X=70, Y=50
Transactions T_1 & T_2 in figure donot follow the two-phase locking protocals because the write_lock(Y) operation follows the unlock(X)

operation in T_2

T_1	T_2
read_lock(y);	
read_item(y);	
unlock(y);	or
	read_lock(x);
	read_item(x);
	unlock(x);
	write_lock(y);
	read_item(y);
	y = x + y;
	write_item(y);
	unlock(y);
write_lock(x);	
read_item(x);	
x := x + y;	
write_item(x);	
unlock(x);	

Transactions T_1 & T_2 in figure follow the two-phase locking protocol. Note that they can produce a deadlock

16] Define schedule? Illustrate with an example?
A schedule is a sequence of operations of multiple transactions that are executed in a concurrent manner while maintaining the consistency of the database

Types of schedules:

1] Serial schedule:
> Transactions execute one after another without interleaving.
> either S = $T_1 \rightarrow T_2 \rightarrow T_3$ or $T_2 \rightarrow T_1 \rightarrow T_3$ etc.
> Always correct but low concurrency

2] Non-serial schedule
> operations of different transactions are interleaved.
> Higher concurrency but may lead to inconsistency.

3] Serializable schedule:
> A non-serial schedule is serializable if its result is equivalent to some serial schedule.
> Two types:
- conflict serializable: if swapping non-conflicting operations gives a serial schedule.
> view serializable: if transactions read the same values and write same final values as in a serial schedule

4] Recoverable schedule.

> If a transaction T1 reads data written by T2, then T2 must commit before T1 commits.

> Prevents dirty read problem.

5] cascadeless schedule.

> NO transaction read uncommitted data from another transaction

> Avoids cascading aborts.

Example:

T1: r(x), w(x), commit

T2: r(x), w(x), commit

Serial: T1 → T2 or T2 → T1

Non-Serial: r1(x), r2(x), w1(x), w2(x), c1, c2

17] Why concurrency control is needed? Demonstrate with an example.

concurrency control techniques are used to ensure the noninterference or isolation property of concurrently executing transactions most of these techniques ensure serializability of schedules using concurrency control protocols that guarantee serializability.

Need for concurrency control.

1. To maintain database consistency.

when multiple transactions execute concurrently

their operations may interface with each other. without control, this can lead to inconsistent database states.

2] TO ensure isolation property of ACID concurrency control ensures that concurrently executing transactions do not interface with each other, Preserving the isolation property.

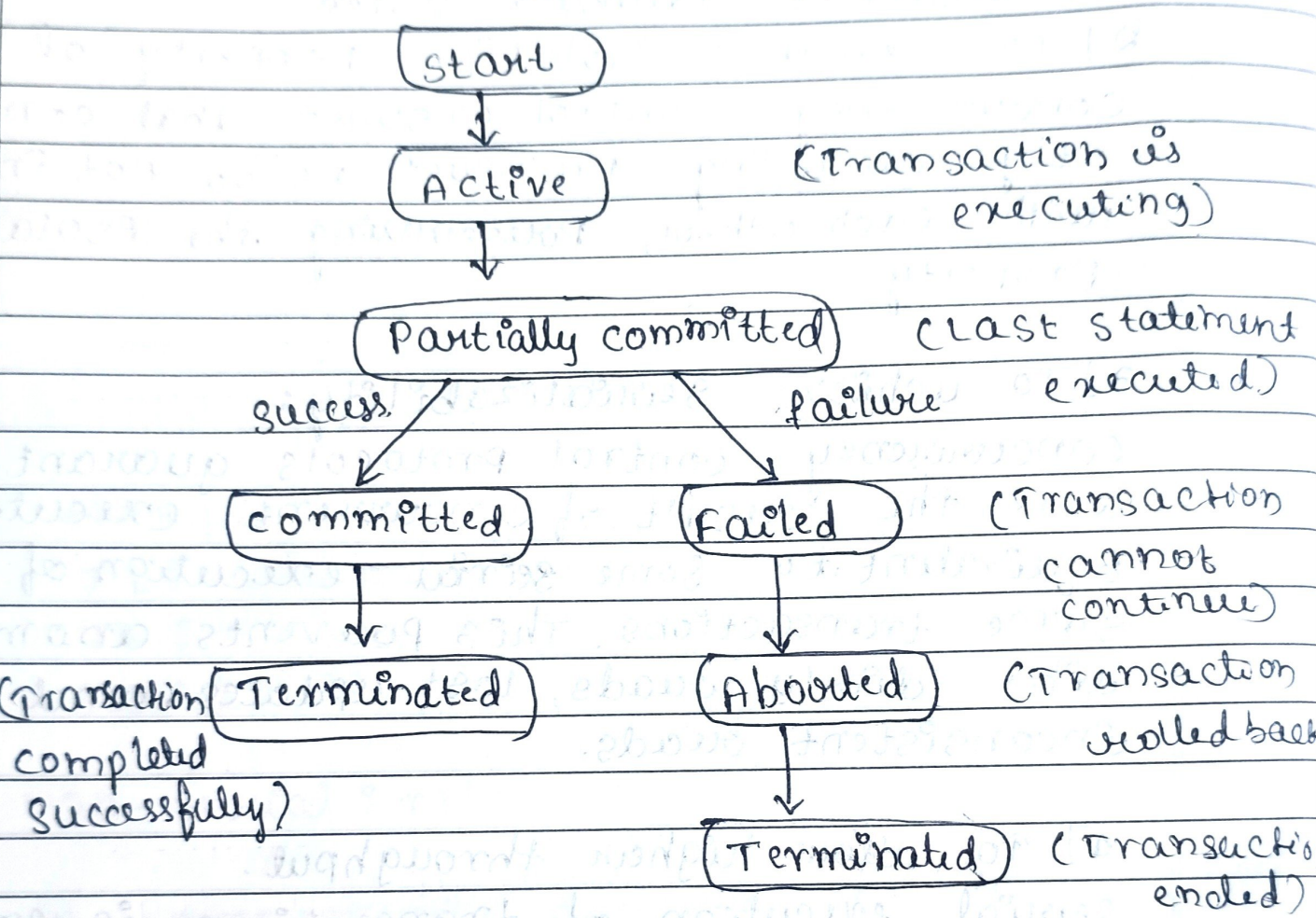
3] To achieve serializability:
concurrency control protocols guarantee that the result of concurrent execution is equivalent to some serial execution of the same transactions. This prevents anomalies like dirty reads, lost updates, and inconsistent reads.

4] To allow higher throughput.
serial execution of transactions is safe but slow. concurrency control allows interleaving of operations from different transactions, improving system performance while still maintaining correctness

example: why concurrency control is needed

T1	T2
read_item(x); //x=100	read_Item(x) //x=100
x = x+50 // x=150	x = x-30 //x=70
write_Item(x)	write_item(x)

18] Demonstrate the database Transaction with transaction digram.



A transaction is logical unit of work that transforms the database from one consistent state to another. it must satisfy ACID properties

→ The transaction begins, enters Active state where operations execute. After the last operation it becomes partially committed. if commit succeeds, it moves to committed and then terminated. if an error occurs, it moves to failed

then Aborted where changes are rolled back using the log, and finally terminated above diagram shows that a transaction either completes fully or is completely undone, ensuring atomicity

19] What is NOSQL? Explain the CAP Theorem

NOSQL systems are distrubuted or non-relational databases designed to handle large volumes of semi-structured, unstructured, or rapidly changing data. They focus on performance, scalability, availability, data replication, Instead of strict consistency and structured schemes used in traditional SQL databases

CAP Theorem

The CAP Theorem, which was originally introduced as the CAP principle, can be used to explain some of the competing requirements in a distributed systems with replication the three letters in CAP refer to three desirable properties of distributed systems with replicated data.

> consistency (among replicated copies)
> availability (of the system for read and write operations) and
> partition tolerance (in the face of the nodes in the system being partitioned by a network fault).

The CAP theorem states that it is not possible to guarantee all three of the desira-ble properties, consistency, Availability and Partition tolerance - at the same time in a distributed system with data replication. if this is the case, then the distributed system designer would have to choose two properties out of the three to guarantee.
It is generally assumed that in many tradi-tional (sql) applications, guaranteeing consistency through the ACID properties is important is important. on other hand, in a NOSQL distributed data store, a weaker consistency level is often acceptable, and guaranteeing the other two properties (availability, partition tolerance is important).

20] what are document based NOSQL systems? basic operations CRUD in mongoDB.
Document-based NOSQL systems:
These systems store data in the form of documents using well-known formats, such as JSON (Javascript object Notation). Documents are accessible via their document id, but can also be accessed rapidly using other accessible indexes.
1) Data model: Data is stored as documents, which are collections of key-value pairs. each document can have different field-no fixed schema

2. Access: Documents are accessible via their document id, but can also be accessed rapidly using other indexes
example: MongoDB, couchDB

Basic operations CRUD in mangoDB

1] Creating a table: create <tablename>, <column family>, <column family>, - - - -

2] Inserting Data: put <tablename>, <rowid>, <dumn family>: <column qualifier>, <value>

3] Reading data (all data in a table): scan <tablname>

4] Retrieve Data (one item): get <table name>, <rowid>

mongoDB uses a JSON-like query language instead of SQL, making it natural for application developers.

21] What is NOSQL GRAPH Database? Explain Neo4j.
Another category of NOSQL system is known as graph databases or graph-oriented NOSQL systems. The data is represented as a graph, which is a collection of vertices (nodes) and edges. Both nodes and edges can be labeled to indicate the type of entities and relationships they represent, and it is generally possible to store data associated with

both individual nodes and individual edges.

Neo4J Data model

Neo4J organizes data using nodes and relationships.

1] Nodes.

> Nodes represents entities.
> Nodes can have properties - key-value pair storing data items.
Example: {Lname: 'smith', Fname: 'John', minit: 'B'}
> Nodes can have labels. Nodes with the same label are grouped into a collection for querying A node can have zero, one or several labels.
Example: EMPLOYEE, DEPARTMENT, PROJECT, LOCATION.

2] Relationships.

- Relationships are directed and connect a start node to an end node
> Each relation has a relationship type which identifies similar relationships.
Example types: WorksFor, Manager, LocatedIn, WorksOn.
> Relationships can be traversed in either direction.

D D M M Y Y Y Y

Properties:

Both nodes and relationships can have properties, specified as a map pattern: { name value } enclosed in curly brackets.

Paths:

A path specifies a traversal of part of the graph. used in queries to specify a pattern. A path is a start node followed by one or more relationships.